VAPT Web Application Scanner

Code, file architecture and workflow, explained in sequence

Project title: **Vulnerability Assessment and Penetration Testing of Web Application**. Repository: github.com/Bunny538/vapt-webapp-scanner. Language: Python. Live demo: bunny538.github.io/vapt-webapp-scanner.

Contents

Part	Topic
1	Problem and how the project answers it (maps to the project slides)
2	Architecture at a glance
3	Repository layout, file by file
4	Data models
5	End-to-end workflow, step by step
6	The thirteen checks
7	Lab target (vulnerable and hardened)
8	Reports, baseline and SARIF
9	Command-line interface
10	Web dashboard and API
11	Tests, CI, Docker and Pages
12	Safety design
13	UML diagrams: use case, class, sequence, collaboration
14	Gap analysis against the proposed system, and roadmap

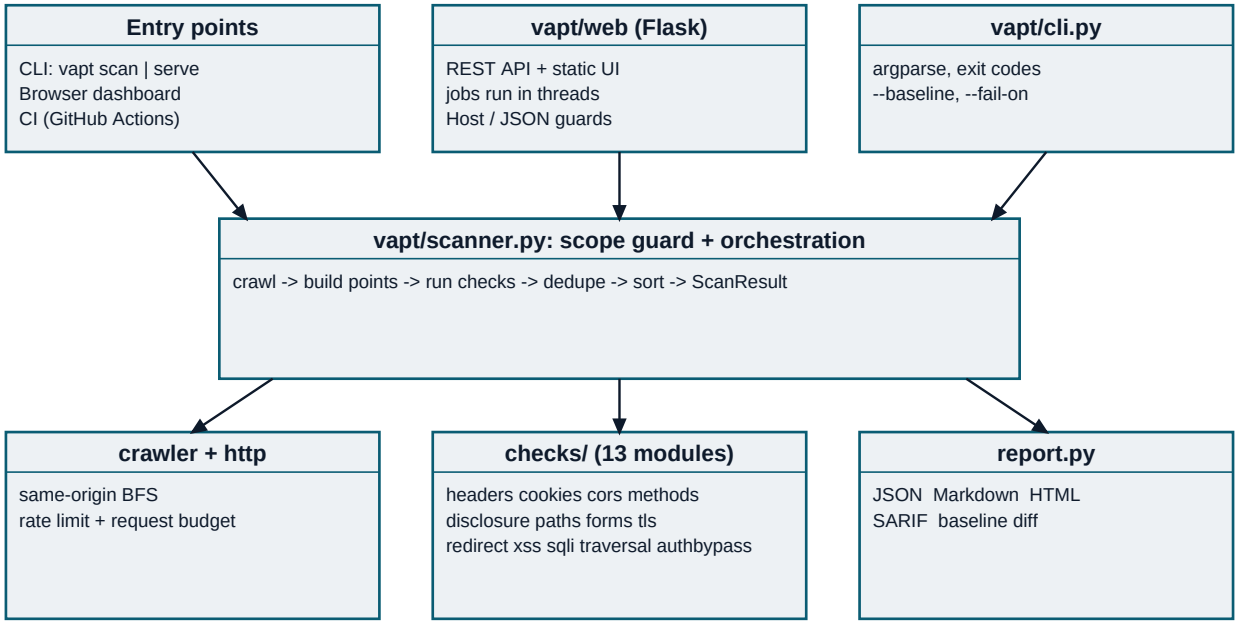
1. Problem and how the project answers it

The project slides state the problem: web applications are attacked through flaws in design or implementation (SQL injection, XSS, broken authentication), and manual testing is slow and error-prone, so a systematic, automated approach is needed. This project is a working Python platform that automates the *assessment* half of VAPT and ships a safe practice target so the results can be demonstrated and verified.

Slide idea	Where it lives in the project
Integrated VAPT platform	vapt scanner + lab target + dashboard in one repository
Automated scanning (SQLi, XSS, auth, misconfiguration)	vapt/checks/*: 13 modules mapped to CWE and OWASP Top 10 (2021)
Simulate real-world attacks	Benign probes: inert XSS marker, single-quote and boolean SQLi, canary-file traversal, foreign-Origin CORS
Detailed reports, prioritized recommendations	report.py: severity-ranked findings with evidence and remediation in JSON, Markdown, HTML, SARIF
Interactive dashboard	vapt/web: severity strip, filters, expandable findings, downloads
DevSecOps integration	Exit codes with --fail-on, --baseline, SARIF for code scanning, GitHub Actions workflows
Customizable test modules	--checks selects modules; a new module is one file plus one registry line
Compliance support	Every finding carries a CWE id and an OWASP Top 10 category
Manual penetration testing	Out of scope for automation: results are labelled for manual verification (see part 14)

2. Architecture at a glance

Three entry points (command line, browser dashboard, CI) all call the same engine, `run_scan()`. The engine enforces the scope guard, crawls, runs the selected checks, de-duplicates and sorts findings, and returns one `ScanResult`. The report layer turns that object into files.



Design principles

- **One engine, many front ends.** CLI, web API and tests share run_scan, so behaviour is identical everywhere.
- **Plug-in checks.** Each check module exposes NAME and run(ctx) and returns Finding objects. One failing check never aborts a scan; its error is recorded.
- **Safe by default.** Scope guard, request budget, rate limit, GET-only injection, benign payloads, no data extraction.
- **Evidence-based findings.** Each finding stores the request evidence, a fix, and CWE/OWASP tags, which reduces false-positive triage time.

3. Repository layout, file by file

```
vapt-webapp-scanner/
  vapt/                    the scanner package
    __init__.py            version string
    __main__.py            enables `python -m vapt`
    cli.py                 argparse CLI: scan, serve, list-checks
    scanner.py             ScanConfig, scope guard, run_scan()
    crawler.py             same-origin crawl, form/parameter discovery
    http.py                Client: rate limit, request budget, timeouts
    models.py              Finding, Form, Point, Page, Context, ScanResult
    report.py              JSON / Markdown / HTML / SARIF, baseline
    utils.py               is_local_host(), snippet()
    checks/                one file per vulnerability class
      __init__.py          registry ALL = {name: module}
      headers cookies cors methods disclosure paths
      forms tls redirect xss sqli traversal authbypass
    web/                   dashboard
      __init__.py          Flask app, jobs, JSON API, security headers
      static/index.html app.css app.js   single-page UI
  lab/app.py               deliberately vulnerable Flask app (+ --hardened)
  tests/                   pytest suite (scanner, lab, web API)
  sample-report/           pre-generated report of the lab scan
  .github/workflows/       ci.yml (tests + lab scan), pages.yml (demo deploy)
  Dockerfile, docker-compose.yml, pyproject.toml, requirements.txt, README.md, LICENSE
```

File	Responsibility
vapt/cli.py	Parses arguments. `scan` builds a ScanConfig, calls run_scan, writes reports, prints a summary, applies --baseline and --fail-on, returns exit code 0, 1 or 2. `serve` starts the dashboard. `list-checks` prints modules.
vapt/scanner.py	Validates the URL, applies the scope guard (local hosts allowed, others need --authorized), validates check names, builds the HTTP client, crawls, builds injectable points, runs each check inside try/except, removes duplicates by (id, url, parameter), sorts by severity, returns ScanResult.
vapt/crawler.py	Breadth-first, same-origin only, limited by max pages and depth. An HTMLParser collects links, forms (action, method, inputs) and HTML comments. Static file extensions are skipped. Set-Cookie headers are captured, including those on redirects. build_points turns query parameters and form fields into Point objects.
vapt/http.py	Wraps requests.Session. Every request increments a counter and stops at the budget; optional delay between requests; fixed User-Agent; optional TLS verification off; custom headers and a cookie for authenticated scans.
vapt/models.py	Dataclasses used everywhere. Context gives checks the client, pages, points and helpers testable_points() and send(point, value).
vapt/report.py	to_json, to_markdown, to_html (all values HTML-escaped), to_sarif (SARIF 2.1.0), write_reports, finding_key and load_baseline.
vapt/utils.py	is_local_host treats loopback, private and link-local addresses as local; snippet extracts evidence around a match.
vapt/web/__init__.py	Flask server: /api/meta, /api/scans (start, list), /api/scans/, report downloads; scans run in background threads; Host and Content-Type guards; strict CSP and other security headers.
vapt/web/static/*	Light-themed single-page UI. Polls the job, shows progress, the severity strip, filters and findings. Falls back to a read-only demo when no API exists (GitHub Pages).
lab/app.py	Practice target with real, intentional flaws; --hardened runs the fixed version for before/after comparison.

4. Data models

Model	Fields and purpose
Finding	id, title, severity (info, low, medium, high, critical), url, description, remediation, evidence, parameter, cwe, owasp. One discovered weakness.
Form	page_url, action, method, inputs[]. Parsed HTML form (never submitted by default).
Point	url, method, params, name, source. One injectable parameter and how to request it.
Page	url, status, headers, text (capped), set_cookies, forms, links, comments. One crawled response.
Context	base, client, pages, points, local, test_post, max_points. Everything a check needs; send() substitutes a payload into a Point without following redirects.
ScanConfig	target, authorized, max_pages, max_depth, delay, timeout, max_requests, test_post, verify_tls, headers, cookie, checks, max_points.
ScanResult	target, timestamps, duration, requests, pages, points, checks_run, findings, errors; summary() counts by severity; max_severity() drives the exit code.

5. End-to-end workflow, step by step

This is the exact order of events for one scan.

Step 1: Start. User runs `vapt scan URL` or presses Start scan in the dashboard. Either way a ScanConfig is built.

Step 2: Validate. scanner.py rejects non-http(s) URLs and unknown check names.

Step 3: Scope guard. If the host is not loopback/private and --authorized was not given, a ScopeError stops the scan (CLI exit code 2; dashboard shows an error).

Step 4: Create client. A Client is created with timeout, delay, request budget, TLS option, headers and cookie.

Step 5: Crawl. crawler.crawl() fetches the start URL, parses HTML, queues same-origin links up to max depth and pages, and records pages, forms, comments and cookies. If nothing is reachable the scan fails with a clear message.

Step 6: Build points. build_points() converts link query parameters and form text fields into Point objects (GET only unless --test-post).

Step 7: Run checks. For each selected check, run(ctx) is called. Passive checks (headers, cookies, forms, tls, disclosure) analyse what was already fetched; active checks (cors, methods, paths, redirect, xss, sqli, traversal) send a few benign requests. Exceptions are caught and listed under errors.

Step 8: Consolidate. Duplicate findings are removed, then sorted by severity, id, URL.

Step 9: Report. write_reports() creates report.json, report.md, report.html (and report.sarif if requested). The dashboard serves the same data over its API.

Step 10: Gate. With --baseline, only findings not present in the older report count. The process exits 1 if any counted finding meets --fail-on (default high), else 0. CI uses this to fail builds.

6. The thirteen checks

Check	How it works	Findings (examples)	Mapping
headers	Reads response headers of the home page	Missing CSP, X-Frame-Options/frame-ancestors, nosniff, Referrer-Policy, Permissions-Policy, HSTS (HTTPS only), weak CSP	A05, CWE-693/1021
cookies	Parses every Set-Cookie	Missing HttpOnly, Secure (HTTPS), SameSite; severity higher for session-like names	A05/A02, CWE-1004/614/1275
cors	Sends a foreign Origin header	Reflected origin with credentials (high), reflected origin, wildcard	A05, CWE-942
methods	OPTIONS request, reads Allow	TRACE, TRACK, PUT, DELETE, CONNECT, PATCH advertised	A05, CWE-650
disclosure	Analyses banners and page bodies	Server/X-Powered-By versions, stack traces, directory listing, sensitive HTML comments	A05, CWE-200/209/548/615
paths	Requests well-known sensitive paths and validates content to avoid soft-404 noise	Exposed .git, .env, backups, db files, phpinfo, server-status, actuator, API docs, robots.txt entries	A05, CWE-538/530/200
forms	Static analysis of parsed forms (no submission)	POST form without CSRF token; password sent over HTTP	A01/A02, CWE-352/319
tls	Opens a TLS connection	No HTTPS, invalid certificate, legacy protocol, certificate expiring within 30 days	A02, CWE-295/326/298
redirect	Injects an external URL into redirect-like parameters (next, url, return...)	Open redirect when Location points to the attacker host	A01, CWE-601
xss	Injects a unique inert marker tag in each parameter	Reflected XSS when the marker returns unencoded in HTML	A03, CWE-79
sqli	Baseline vs single quote vs TRUE/FALSE conditions	Error-based SQLi (database error text) and likely boolean-based SQLi; no data extracted	A03, CWE-89
authbypass	With --test-post: submits an unknown user plus an always-true SQL condition to login forms and compares with a normal failed login	Authentication bypass via SQL injection on a login form (the attack shown in the project screenshots)	A03, CWE-89
traversal	Sends traversal sequences to file-like parameters and looks for canary signatures	Arbitrary file read (CWE-22); only well-known system file markers are matched	A01, CWE-22

Severity scale: critical, high, medium, low, info. Heuristic detection is expected to yield false positives and negatives; the report footer says so and every finding includes evidence so a human can verify it.

Adding a new check

```
# vapt/checks/mycheck.py
from ..models import Finding
NAME = "mycheck"
def run(ctx):
    r = ctx.client.get(ctx.base + "/something")
    return [Finding("VAPT-MY-001", "Title", "low", ctx.base, "What is wrong", "How to fix")] if r and r.ok
# then register it in vapt/checks/__init__.py
```

7. Lab target (vulnerable and hardened)

lab/app.py is a small Flask application used as a legal practice target. In normal mode it contains intentional flaws: reflected XSS in /search, SQL injection in /user (in-memory SQLite), open redirect in /redirect, path traversal in /download (against a simulated file store, never the real filesystem), reflected CORS origin, missing headers and cookie flags, risky HTTP methods, exposed /.git/HEAD, /.env, /backup.sql, /robots.txt, directory listing, debug traces, and a sensitive HTML comment. With --hardened every issue is fixed (output encoding, parameterised SQL, redirect validation, allow-listed downloads, security headers, cookie flags, no exposed files). Account flow (matches the project screenshots): Home, New User Signup (Register.html), User Login (UserLogin.html), welcome page and "login failed" page, using the project title and blue theme. In normal mode the login query is built by string concatenation, so an unknown user with a password such as `' or '1'='1` is logged in (SQL injection / authentication bypass); with --hardened the query is parameterised and passwords are hashed. The tests assert the scanner finds every expected flaw in the vulnerable build and no medium-or-higher finding in the hardened build, which demonstrates before and after remediation.

8. Reports, baseline and SARIF

- **JSON**: complete machine-readable result including the severity summary.
- **Markdown**: readable report with summary table and per-finding details.
- **HTML**: self-contained page; all scanned data is escaped so evidence cannot execute.
- **SARIF 2.1.0**: lets GitHub code scanning and other tools ingest findings.
- **Baseline**: --baseline report.json ignores findings already known (matched by id, URL, parameter) so CI fails only on new problems.

9. Command-line interface

```
vapt scan <url> [--authorized] [--checks xss,sqli,...] [--max-pages 25] [--max-depth 2]
                [--max-requests 500] [--delay 0.2] [--timeout 10] [--test-post] [--insecure]
                [--header 'Name: value'] [--cookie 'sid=...'] [--out reports]
                [--format json,md,html,sarif] [--baseline report.json]
                [--fail-on {critical,high,medium,low,info,none}]
vapt serve [--host 127.0.0.1] [--port 8080]
vapt list-checks
Exit codes: 0 clean, 1 finding at/above --fail-on, 2 usage or scope error
```

10. Web dashboard and API

`vapt serve` starts a Flask server (loopback by default). The browser UI is a single page of HTML, CSS and JavaScript.

Endpoint	Purpose
GET /	The single-page UI
GET /api/meta	Version and the list of checks
POST /api/scans	Validate options, create a job, run run_scan in a background thread, return the job id
GET /api/scans	Recent scans (summaries)
GET /api/scans/	Status (queued, crawling, running, done, error), progress, log, and the full result when done
GET /api/scans//report.	Download json, md, html or sarif

How the UI works

- The form posts JSON; the page then polls the job every 0.6 s and shows crawl progress and which check is running.
- When done, a **severity strip** draws one coloured segment per finding. Clicking a segment opens that finding; the legend filters by severity; a text box filters by title, URL, CWE or parameter.
- Each finding expands to description, evidence, fix and tags. All text is inserted with textContent, never as HTML.
- If no API exists (GitHub Pages) the page loads demo/report.json and becomes a read-only demo.

Server protections

- Requests whose Host header is not localhost are refused (stops DNS rebinding).
- POST requires a JSON content type (stops simple cross-site form posts).
- Strict Content-Security-Policy, X-Frame-Options DENY, nosniff, no-referrer.
- Scan limits are clamped server-side; the same scope guard applies as in the CLI.

11. Tests, CI, Docker and Pages

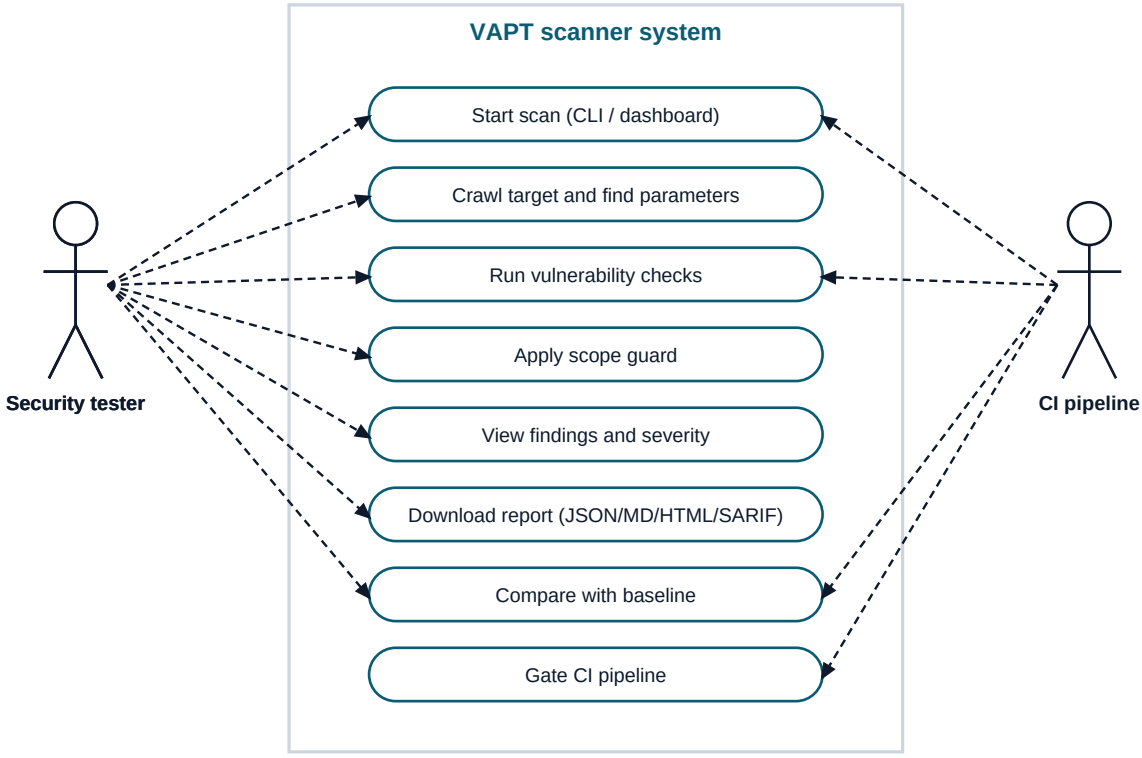
- **tests/test_scanner.py**: detects every expected lab vulnerability; hardened lab has none at medium or above; POST not fuzzed by default; request budget enforced; scope guard; bad input; reports render and escape; CLI exit codes; SARIF; baseline.
- **tests/test_web.py**: index and meta, a full scan through the API with report downloads, scope error reporting, input validation and Host/Content-Type guards.
- **conftest.py**: starts the vulnerable and hardened labs in-process on random ports.
- **ci.yml**: runs tests on Python 3.10 to 3.12, then scans the lab and uploads reports as a build artifact.
- **pages.yml**: assembles the demo (UI + sample report) and deploys to GitHub Pages on every push to main.
- **Docker**: one image for both lab and scanner; docker compose starts the lab bound to 127.0.0.1 only and scans it, writing ./reports.

12. Safety design

Control	Effect
Scope guard	Non-private targets are refused unless --authorized (or the confirmation box in the UI) is given
Request budget and delay	Hard cap on total requests and optional rate limiting
Benign payloads	Inert marker tags, a single quote, canary files; no exploitation or data extraction
GET only by default	POST forms are fuzzed only with --test-post because it may change state
Same-origin crawl	The crawler never leaves the target origin
Escaped output	Reports and UI treat scanned data as text
Local dashboard	Binds to loopback, Host and content-type guards

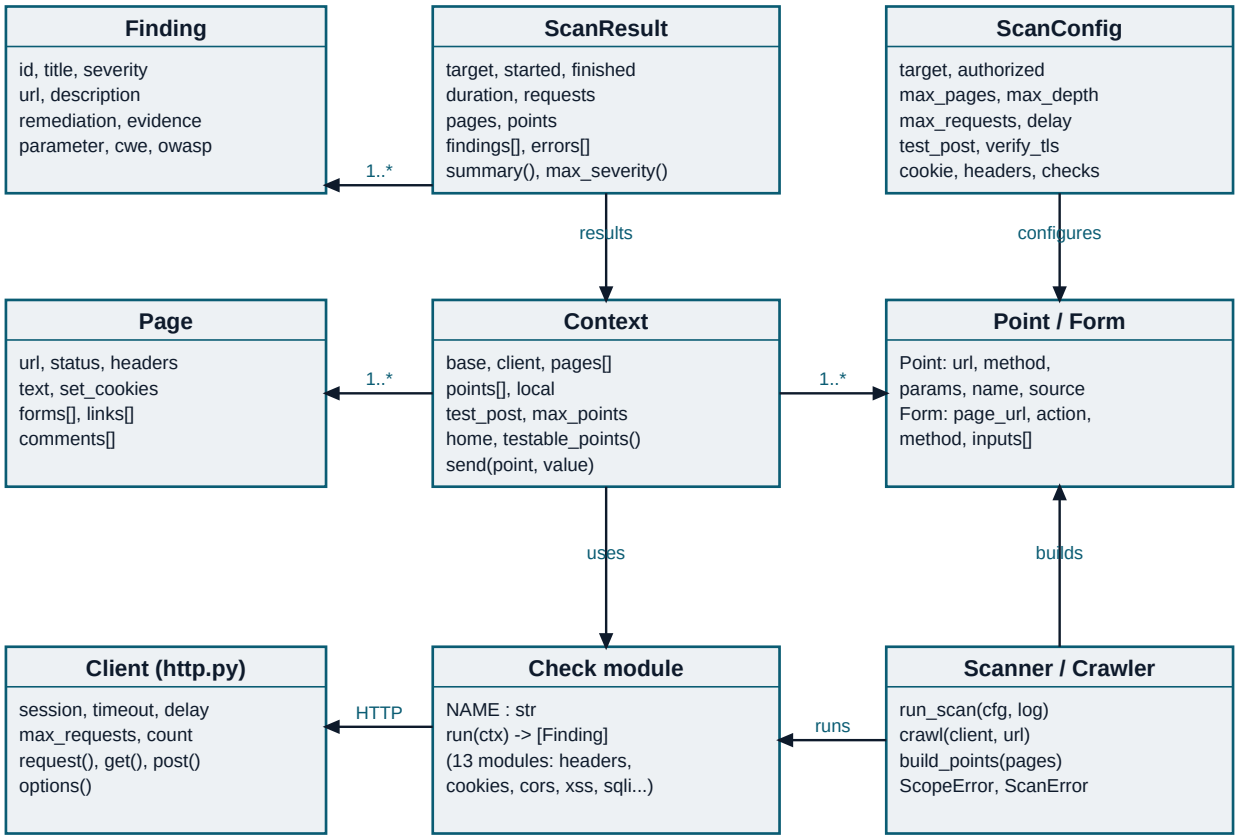
13. UML diagrams

13.1 Use case diagram



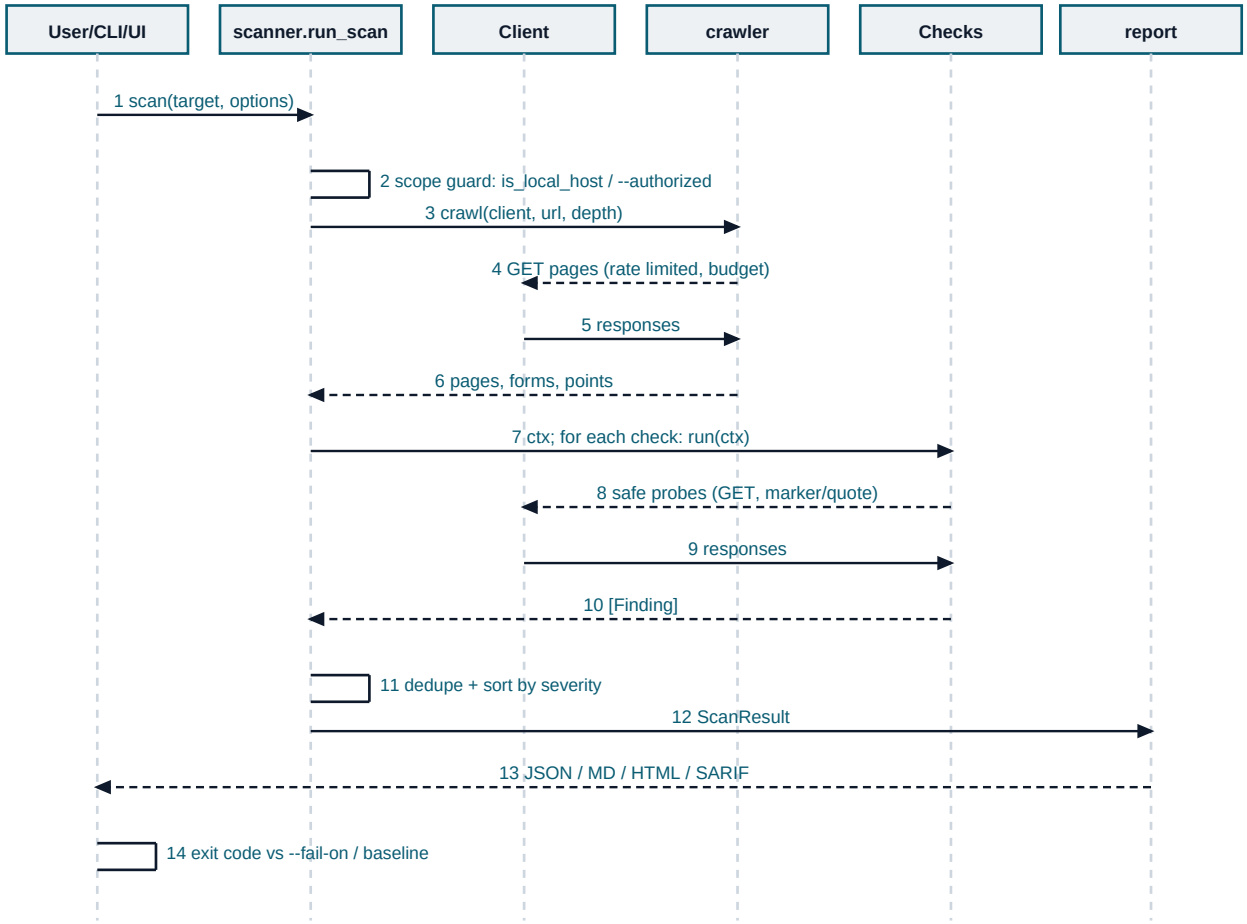
Actors: the security tester (CLI or browser) and the CI pipeline. The scope guard is part of every scan, so it is shown as its own use case.

13.2 Class diagram

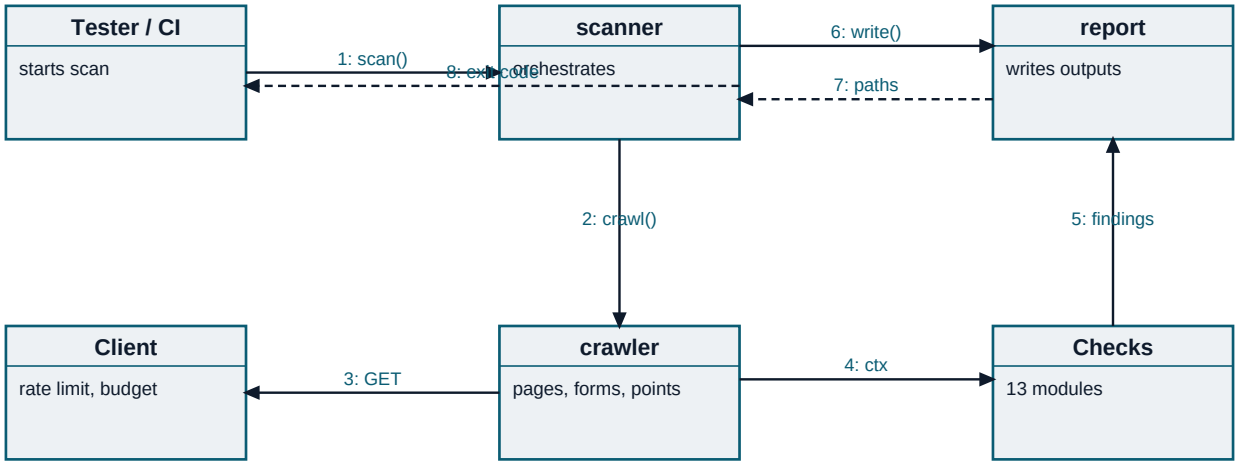


Context aggregates Pages and Points and gives every check controlled access to the Client. Each check module returns Findings; ScanResult collects them.

13.3 Sequence diagram



13.4 Collaboration diagram



Numbers show the order of messages between the objects: scan, crawl, HTTP fetches, context hand-off, findings, report writing and exit code.

14. Gap analysis against the proposed system, and roadmap

The project documents propose a platform that also uses machine learning, continuous monitoring and manual testing. This table is an honest comparison of what exists today.

Proposed feature	Status	Notes
Automated scanning of injection, XSS, misconfiguration	Done	13 checks, CWE/OWASP mapped
Detailed reports and visual dashboard	Done	HTML/MD/JSON/SARIF and the web dashboard
Risk prioritization with severity levels	Partly	Five severity levels and ordering. No numeric risk score or exploitability weighting yet
DevSecOps / CI integration	Done	Exit codes, baseline, SARIF, GitHub Actions
Customizable test modules	Done	--checks and plug-in module API
Authentication and session flaws	Partly	Login-form SQL injection bypass (authbypass check), cookie flags, CSRF tokens, password over HTTP, authenticated scans via cookie/header. No brute-force or session-fixation tests
Real-time or continuous scanning with alerts	Not yet	Scans run on demand or in CI; scheduled runs and notifications are a next step
Machine-learning false-positive reduction	Not yet	Needs labelled scan data; a rule-based confidence score is a sensible first step
Zero-day and threat-intelligence feeds	Not yet	Would require integration with CVE/NVD feeds
Manual penetration testing and business-logic testing	Out of scope	Cannot be automated; the report labels results for manual verification

Suggested next steps

- Add a numeric risk score per finding (severity, confidence, exposure) and sort by it.
- Add a scheduler and email/webhook alerts for new findings (builds on --baseline).
- Add a confidence field so uncertain heuristics are separated from confirmed issues.
- Add more checks: SSRF, security.txt, subresource integrity, session fixation.